

Guía para Dummies — Lab 02
Scripts de Shell, Packet Tracer, VIM y SAMBA

Camilo Aguirre y Juan David Rangel

Arquitectura y Servicios de Red (AYSR)
Escuela Colombiana de Ingeniería Julio Garavito

Bogotá, D.C., 21 de agosto de 2026

Índice general

Antes de empezar	2
0.1. Scripts de shell (versión simple)	2
0.1.1. 1.1 mi_ls.sh — listar con menú	2
0.1.2. 1.2 buscar.sh — buscar archivos o palabras	2
0.1.3. 1.3 revisar_logs.sh — ver los logs	3
0.1.4. 1.4 newgroup.sh — crear grupo	3
0.1.5. 1.5 newuser.sh — crear usuario	4
0.2. Comandos de Packet Tracer (CLI de routers y switches)	4
0.3. Comandos de VIM (el editor)	5
0.4. Comandos de SAMBA / SMB	5
Verificado en el lab (21/08/2026)	6
0.5. Carpetas de Linux (qué son y qué traen)	7
Dónde está todo	8

Antes de empezar

Un **script** es un archivo de texto con comandos que la terminal ejecuta uno detrás de otro, como una receta.

Comando	Qué es	Ejemplo
<code>#!/bin/bash</code>	Primera línea SIEMPRE: dice “ejecutame con bash”	<code>#!/bin/bash</code>
<code># texto</code>	Comentario: la máquina lo ignora, es para vos	<code># nota</code>
<code>echo "..."</code>	Escribe texto en pantalla	<code>echo "hola"</code>
<code>read -p "..."</code>	Pregunta y espera que escribas; guarda en <code>x</code>	<code>read -p</code> <code>.°pcion: .°p</code>
<code>\$x</code>	“El valor que guardaste en <code>x</code> ”	<code>\$nombre</code>
<code>if/then/else/fin</code>	“Si pasa esto... hacé esto... si no... fin”	
<code>["\$x" = "1"]</code>	Compara si <code>x</code> es igual a “1” (los espacios importan)	
<code>case/;;/esac</code>	Menú: según el número, hace algo distinto	
<code>2>/dev/null</code>	Esconde los mensajes de error	
<code> </code>	“Si lo anterior falló, entonces...”	

0.1. Scripts de shell (versión simple)

0.1.1. 1.1 mi_ls.sh — listar con menú

```
#!/bin/bash
# Muestra un directorio de 3 formas distintas

read -p "Que directorio? " dir

echo "1) Normal    2) Con detalle    3) Con ocultos"
read -p "Opcion: " op

case $op in
  1) ls "$dir" ;;
  2) ls -l "$dir" ;;
  3) ls -a "$dir" ;;
esac
```

- `read -p "..."``dir`: pregunta y guarda la respuesta en la variable `dir`.
- `case $op in`: menú según lo que eligió; cada caso termina en `;;` y el menú cierra con `esac`.
- `ls -l`: listado con detalle (permisos, dueño, tamaño, fecha). `ls -a`: incluye archivos ocultos.

0.1.2. 1.2 buscar.sh — buscar archivos o palabras

```
#!/bin/bash
# Busca un archivo por nombre o una palabra dentro de un archivo

echo "1) Buscar archivo por nombre    2) Buscar palabra en archivo"
read -p "Opcion: " op

if [ "$op" = "1" ]; then
  read -p "Nombre (o parte): " nombre
  find . -name "$nombre" 2>/dev/null
elif [ "$op" = "2" ]; then
  read -p "En que archivo? " archivo
```

```

read -p "Que palabra? " palabra
grep -n "$palabra" "$archivo"

else
    echo "Opcion no valida"
fi

```

- `if [ "$op- "1"]`: compara la elección (los espacios dentro de `[ ]` son obligatorios).
- `find . -name "$nombre"`: busca archivos cuyo nombre CONTENGA lo escrito (los `*` son comodín); `2>/dev/null` esconde errores de permisos.
- `grep -n "$palabra$archivo"`: busca la palabra DENTRO del archivo y `-n` muestra el número de línea.
- `elif`: “si no fue 1 pero sí 2”; `else`: cualquier otra cosa; `fi`: cierra.

0.1.3. 1.3 revisar _logs.sh — ver los logs

```

#!/bin/bash
# Muestra las ultimas 15 lineas de los 3 logs del sistema

tail -15 /var/log/syslog
echo "-----"
tail -15 /var/log/messages
echo "-----"
tail -15 /var/log/secure 2>/dev/null || echo "(no hay log secure)"

```

- `tail -15`: muestra el FINAL del archivo (lo más reciente), las últimas 15 líneas.
- `2>/dev/null || echo "(no hay log secure)"`: si el log no existe, esconde el error y avisa (no rompe).

0.1.4. 1.4 newgroup.sh — crear grupo

```

#!/bin/bash
# Crea un grupo si todavia no existe

read -p "Nombre del grupo: " nombre

if getent group "$nombre" > /dev/null; then
    echo "Ese grupo ya existe"
else
    groupadd "$nombre"
    echo "Grupo $nombre creado"
fi

```

- `getent group "$nombre" > /dev/null`: le pregunta al sistema si el grupo ya existe (la respuesta se esconde; solo importa si acertó).
- `groupadd`: creación real del grupo.

0.1.5. 1.5 newuser.sh — crear usuario

```
#!/bin/bash
# Crea un usuario con su grupo y su carpeta home

read -p "Usuario: " usuario
read -p "Grupo: " grupo

groupadd "$grupo" 2>/dev/null

useradd -m -g "$grupo" "$usuario"

passwd "$usuario"

echo "Listo: $usuario en el grupo $grupo"
```

- `groupadd "$grupo" 2>/dev/null`: asegura que el grupo exista (error de duplicado escondido, sigue sin romper).
- `useradd -m -g "$grupo$usuario"`: `-m` crea la carpeta home automáticamente; `-g` lo mete en el grupo indicado.
- `passwd "$usuario"`: pide la contraseña de forma interactiva.

0.2. Comandos de Packet Tracer (CLI de routers y switches)

En PT se abre un router/switch y se va a la pestaña **CLI**. Todo arranca así:

Comando	Qué hace
<code>enable</code>	Entra al modo privilegiado (modo admin); el prompt cambia a Router#
<code>configure terminal</code>	Entra al modo de configuración; prompt: Router(config)#
<code>hostname X</code>	Le pone nombre al dispositivo (ej. <code>hostname Router0</code>)
<code>interface g0/0</code>	Entra a la configuración de un puerto; prompt: Router(config-if)#
<code>ip address 10.0.10.1 255.255.255.0</code>	Asigna IP y máscara a la interfaz
<code>no shutdown</code>	PRENDE la interfaz (por defecto está apagada)
<code>clock rate 64000</code>	En el cable serial: el extremo DCE marca la velocidad
<code>exit</code>	Sale un nivel (de la interfaz a config global)
<code>end</code>	Sale de todo el modo config de una vez
<code>ping 10.0.20.10</code>	Prueba de conectividad hacia otra IP
<code>show ip interface brief</code>	Muestra las interfaces con IP y estado (Up/Down)
<code>show running-config</code>	Muestra la configuración actual del dispositivo
<code>write</code>	GUARDA la config (equivale a <code>copy running-config startup-config</code>)
<code>show version</code>	Versión de IOS, hardware y tiempo encendido

Modo simulación (para ver los paquetes):

1. Clic en el botón **Simulation** (abajo a la derecha).

2. Clic en **Edit Filters** → marcar solo **ICMP** (y ARP para ver la resolución de MAC).
3. Ir a un server → **Desktop** → **Command Prompt** → ping <IP destino>.
4. Los eventos aparecen en la lista; clic en **Auto Capture / Play**.
5. Clic en un paquete → **PDU Information**: encapsulación capa por capa (Ethernet → IP → ICMP).

0.3. Comandos de VIM (el editor)

VIM tiene 3 modos: **Normal** (navegar, borrar, copiar — es el inicial), **Inserción** (escribir; se entra con **i** o **a**, se sale con **Esc**) y **Comando** (guardar/salir/buscar; se entra con **:**).

Comando	Qué hace
vi archivo.txt	Abre el editor con ese archivo
i	Modo inserción: se escribe ANTES del cursor
a	Modo inserción: se escribe DESPUÉS del cursor (append)
Esc	Vuelve al modo Normal
x	Borra el carácter bajo el cursor
dw	Borra una palabra (d =delete, w =word)
dd / 5dd	Borra la línea / 5 líneas
u / Ctrl+r	Deshace / rehace
yy / p	Copia la línea (yank) / pega después (P mayúscula antes)
G / 5G	Va a la última línea / a la línea 5
gUU	La línea actual a MAYÚSCULAS
/palabra	Busca hacia adelante; n = siguiente coincidencia
:w	Guarda sin salir (w =write)
:q / :wq / :q!	Sale / guarda y sale / sale SIN guardar (el ! fuerza)
:1,4s/a/-/g	Reemplaza en las líneas 1–4: s =substitute, g =todas las de cada línea
:%s/al/##/g	Reemplaza en TODO el archivo (% = todo el archivo)
:13,16d	Borra las líneas 13 a 16
:set number	Muestra los números de línea

0.4. Comandos de SAMBA / SMB

Conceptos: **SMB** es el protocolo de Windows para compartir archivos en la red. **SAMBA** es la implementación libre para que un sistema Unix “hable Windows”. Un **share** es la carpeta publicada. El protocolo escucha en el **puerto 445**. Los usuarios SMB (**smbpasswd**/**tdbsam**) tienen contraseña APARTE de la del sistema.

Servidor (en Solaris):

Comando	Qué hace
<code>pkg list samba</code>	Confirma el paquete instalado
<code>svcadm enable svc:/network/smb/server</code>	Enciende el servidor SMB nativo (SMF)
<code>netstat -an grep .445</code>	Verifica que el puerto 445 esté en LISTEN
<code>smbpasswd -a claudia</code>	Agrega usuario a la base SMB (-a=add); la contraseña SMB es INDEPENDIENTE de la del sistema
<code>pdbedit -L</code>	Lista los usuarios SMB registrados
<code>/usr/sbin/smbd -D</code>	Levanta el daemon Samba en segundo plano (-D=daemon)
<code>testparm</code>	Valida la sintaxis de <code>/etc/samba/smb.conf</code>
<code>smbstatus</code>	Muestra quién está conectado (sesiones activas)

Cliente (Slackware y Windows):

Comando	Qué hace
<code>smbclient -L //IP -U claudia</code>	-L lista los shares del servidor (el “menú”)
<code>smbclient //IP/compartido -U usuario -c 'put x; get y'</code>	-c ejecuta sin entrar a la consola: put sube, get baja
<code>net use \\IP\compartido /user:claudia clave</code>	En Windows: mapea el recurso (ruta UNC)
<code>dir \\IP\compartido</code>	En Windows: ver el contenido del share

Configuración clave de `/etc/samba/smb.conf`:

Parámetro	Qué hace
<code>workgroup = WORKGROUP</code>	Grupo de trabajo estilo Windows al que se anuncia
<code>security = user</code>	Cada conexión exige usuario y contraseña (nada anónimo)
<code>passdb backend = smbpasswd</code>	Las contraseñas SMB se guardan en la base de usuarios SMB propia (la de nuestro Solaris)
<code>valid users = claudia admin</code>	Lista blanca: SOLO estos usuarios entran
<code>writable = yes</code>	Se puede escribir en el share
<code>path = /export/compartido</code>	La carpeta REAL del disco que se comparte
<code>create mask = 0660</code>	Permisos de archivos nuevos (dueño y grupo rw, otros nada)
<code>directory mask = 0770</code>	Permisos de carpetas nuevas (dueño y grupo rwx, otros nada)

Verificado en el lab (21/08/2026)

VIM: se probaron todos los comandos de la tabla con un archivo de prueba y con el `himno.txt` real del lab: `x`, `dw`, `dd`, `u`, `redo` (`Ctrl+r`), `yy+p`, `G`, `5G`, `gUU`, `/Escuela`, `:1,4s/a/-/g`, `:%s/al/##/g` (reemplazó en 3 líneas, igual que la evidencia), `:13,16d`, `:1,5d`, `:w`, `:q`, `:wq`, `:q!` y `:set number`. El borrado con conteo (`5dd`) se confirmó por equivalencia: 5 borrados seguidos reducen el archivo de 7 a 2 líneas, igual que `:1,5d`.

SAMBA: en Solaris se verificaron `pkg list samba` (4.7.6), `testparm` (config válida), `netstat -an | grep .445` (LISTEN), `smbstatus` (sesión activa de Windows del 17/08), `pdbedit -L` (claudia, admin) y `smbpasswd -a/-x` con un usuario temporal (creado y eliminado). Desde

Slackware: `smbclient -L //192.168.1.11` lista el share `compartido` con los archivos reales del lab, y `put/get` funcionaron ida y vuelta.

0.5. Carpetas de Linux (qué son y qué traen)

Carpeta	Qué es y qué trae	Ejemplo de nuestro lab
<code>/bin</code>	Binarios esenciales: <code>ls</code> , <code>cat</code> , <code>grep</code> , <code>find</code> , <code>bash</code>	los usamos en los scripts
<code>/sbin</code>	Binarios de administración: <code>ifconfig</code> , <code>mount</code>	<code>sudo</code> para ejecutar como root
<code>/etc</code>	Configuración del sistema y servicios	<code>smb.conf</code> de SAMBA y <code>fstab</code> (buscado con <code>buscar.sh</code>)
<code>/var/log</code>	Logs del sistema: <code>syslog</code> , <code>messages</code> , <code>secure</code>	revisados con <code>revisar_logs.sh</code> (filtro <code>sshd</code>)
<code>/home</code>	Carpeta de cada usuario normal	el <code>newuser.sh</code> crea homes ahí
<code>/root</code>	Carpeta del administrador	nuestros scripts en <code>/root/scripts/</code>
<code>/tmp</code>	Archivos temporales (se borran al reiniciar)	el <code>put</code> de prueba: <code>/tmp/prueba-lab02.txt</code>
<code>/usr</code>	Software instalado y librerías	<code>/usr/sbin/smbd</code> (daemon de Samba)
<code>/dev</code>	Hardware representado como archivos (discos, terminales)	<code>/dev/sda3</code> en el <code>fstab</code> que grepeamos
<code>/proc</code>	Información del kernel y procesos en vivo	—
<code>/boot</code>	Kernel e imágenes de arranque	—
<code>/mnt /media</code>	Puntos de montaje de discos extraíbles/red	(en Slackware el kernel no tiene cifs)
<code>/opt</code>	Software de terceros	—
<code>/run</code>	PID y sockets de procesos activos	—
<code>/srv</code>	Datos de servicios	el share está en <code>/export</code> (Solaris)
<code>/var</code>	Datos variables: logs, colas, cachés	los logs de arriba

Regla rápida: el nombre en inglés dice lo que guarda: `etc` = configuración, `var/log` = lo que

varía (logs), **home** = casas de usuarios, **bin** = binarios (ejecutables), **dev** = devices (dispositivos).

Dónde está todo

- **VM Slackware** (la del lab): scripts en `/root/scripts/` (los originales del 17/08 quedaron en `/root/scripts/originales-17-08/`).
- **Repo GitHub**: `scripts-lab02/simples/` (scripts) y esta guía.